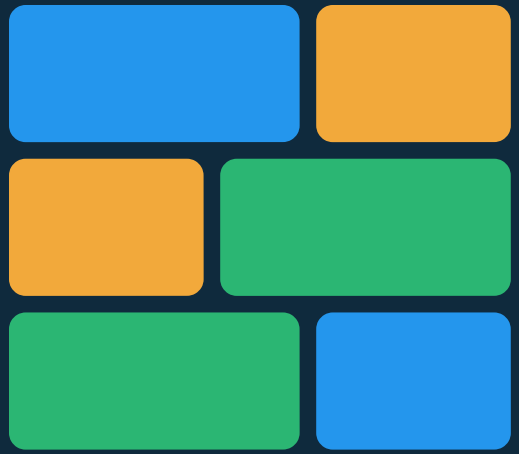
Docker para Full Stack

Entorno y despliegue · DSY1104 Full Stack 2



Qué vamos a aprender

1. **Entorno**
   Instalar y verificar Docker
2. **Conceptos**
   Imagen, contenedor, volumen y red
3. **Backend**
   Dockerfile de Spring Boot
4. **Frontend**
   Dockerfile de React con Nginx
5. **Compose**
   Tres servicios con un comando
6. **Despliegue**
   Publicar y ejecutar en un servidor

Por qué usar Docker

“En mi máquina funciona”

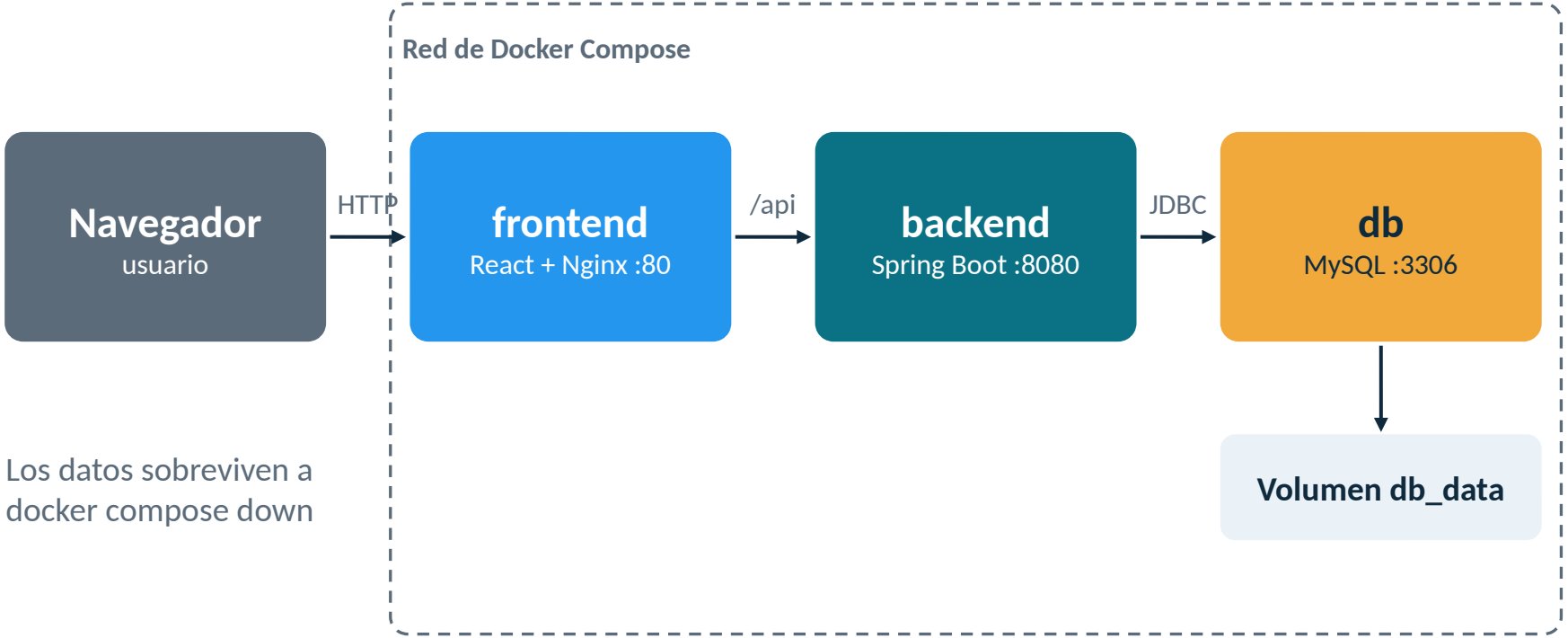
Con Docker funciona en cualquier máquina que tenga Docker.

1. **Mismo entorno en todas partes**
   Tu PC, la de tu profesor y el servidor usan la misma imagen.
2. **Un comando levanta todo**
   Frontend, backend y base de datos con docker compose up.
3. **Despliegue repetible**
   Lo que probaste es exactamente lo que se publica.

Cinco conceptos clave

Imagen	Plantilla de solo lectura: sistema base, dependencias y tu aplicación
Contenedor	Instancia en ejecución de una imagen, aislada del resto
Registry	Repositorio de imágenes: Docker Hub, GitHub Container Registry
Volumen	Almacenamiento que sobrevive al contenedor: datos de MySQL
Red	Conecta contenedores por nombre: el backend llega a la base como db

Arquitectura de la solución



Los datos sobreviven a docker compose down

Backend: Dockerfile multi-stage

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

Etapa 1

Maven compila el proyecto

Etapa 2

Solo el jar sobre un JRE

Caché

El pom se copia primero

Frontend: React servido por Nginx

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
```

Dockerfile

```
location / {
  try_files $uri /index.html;
}

location /api/ {
  proxy_pass http://backend:8080/;
}
```

nginx.conf

Etapa 1 compila con Node; etapa 2 sirve con Nginx

Las variables VITE_* se incrustan al compilar

Variables de entorno y secretos

```
DB_NAME=appdb
DB_USER=appuser
DB_PASSWORD=cambia_esta_clave
DB_ROOT_PASSWORD=cambia_esta_clave_root
BACKEND_PORT=8080
FRONTEND_PORT=80
```

Archivo .env (no se sube a Git)

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
```

application.properties lee las variables

Plantilla en Git

.env.example sí se sube

Nada en la imagen

Sin claves en el Dockerfile

Claves distintas

Desarrollo y producción separados

Docker Compose: tres servicios

```
services:
  db:
    image: mysql:8.4
    volumes: ["db_data:/var/lib/mysql"]
  backend:
    build: ./backend
    environment:
      DB_URL: jdbc:mysql://db:3306/${DB_NAME}
    depends_on:
      db: { condition: service_healthy }
  frontend:
    build: ./frontend
    ports: ["80:80"]
volumes:
  db_data:
```

Nombre = host

El backend llega a db por su nombre

service_healthy

Espera a que MySQL esté listo

Volumen

Los datos sobreviven al down

Despliegue en un servidor

1

Publicar imágenes

docker build y docker push con etiqueta de versión

2

Preparar el servidor

Docker, Compose, el .env de producción y el puerto 80 abierto

3

Ejecutar

pull y up -d con el compose de producción

```
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

Producción publica solo el puerto 80; la base de datos no se expone.

Comandos que usarás a diario

Comando	Qué hace
`docker compose up -d --build`	Construye y levanta en segundo plano
`docker compose ps`	Muestra el estado de los servicios
`docker compose logs -f backend`	Sigue los logs del backend
`docker compose exec backend sh`	Abre una terminal dentro del contenedor
`docker compose down`	Apaga y conserva los datos
`docker compose down -v`	Apaga y borra el volumen (pierdes datos)

Errores frecuentes

Síntoma	Causa probable	Solución
Connection refused a la base	URL con localhost o arranque anticipado	Usar db como host y healthcheck
Port is already allocated	Puerto ocupado en tu PC	Cambiar el puerto en .env
Access denied for user	Claves cambiadas con el volumen ya creado	down -v y volver a levantar
404 al recargar en React	Falta try_files en Nginx	Agregar try_files a nginx.conf
No se ven los cambios	Caché de capas	build --no-cache y up -d

Actividad práctica

A · Contenerizar

Dockerfile multi-stage en backend y frontend

B · Orquestar

Compose con db, backend, frontend y volumen

C · Desplegar

Imágenes publicadas y app corriendo en un servidor

D · Documentar

README para que otra persona la levante sola

Prueba final de calidad

```
git clone <URL_DEL_REPOSITORIO>
cd dsy1104-docker-fullstack
cp .env.example .env
docker compose up -d --build
```

Si otra persona lo logra solo con tu README, tu entorno está bien armado.

Aplicación en http://localhost · API en http://localhost:8080/tareas